
chapter

3

BASIC INPUT/OUTPUT

CHAPTER OBJECTIVES

In this chapter you will learn about:

- Transferring data between a processor and input/output (I/O) devices
- The programmer's view of I/O transfers
- How program-controlled I/O is performed using polling
- How interrupts are used in I/O transfers

One of the basic features of a computer is its ability to exchange data with other devices. This communication capability enables a human operator, for example, to use a keyboard and a display screen to process text and graphics. We make extensive use of computers to communicate with other computers over the Internet and access information around the globe. In other applications, computers are less visible but equally important. They are an integral part of home appliances, manufacturing equipment, transportation systems, banking, and point-of-sale terminals. In such applications, input to a computer may come from a sensor switch, a digital camera, a microphone, or a fire alarm. Output may be a sound signal sent to a speaker, or a digitally coded command that changes the speed of a motor, opens a valve, or causes a robot to move in a specified manner. In short, computers should have the ability to exchange digital and analog information with a wide range of devices in many different environments.

In this chapter we will consider the input/output (I/O) capability of computers as seen from the programmer's point of view. We will present only basic I/O operations, which are provided in all computers. This knowledge will enable the reader to perform interesting and useful exercises on equipment found in a typical teaching laboratory environment. More complex I/O schemes, as well as the hardware needed to implement the I/O capability, are discussed in Chapter 7.

3.1 ACCESSING I/O DEVICES

The components of a computer system communicate with each other through an interconnection network, as shown in Figure 3.1. The interconnection network consists of circuits needed to transfer information between the processor, the memory unit, and a number of I/O devices.

In Chapter 2, we described the concept of an address space and how the processor may access individual memory locations within such an address space. Load and Store instructions use addressing modes to generate effective addresses that identify the desired locations. This idea of using addresses to access various locations in the memory can be

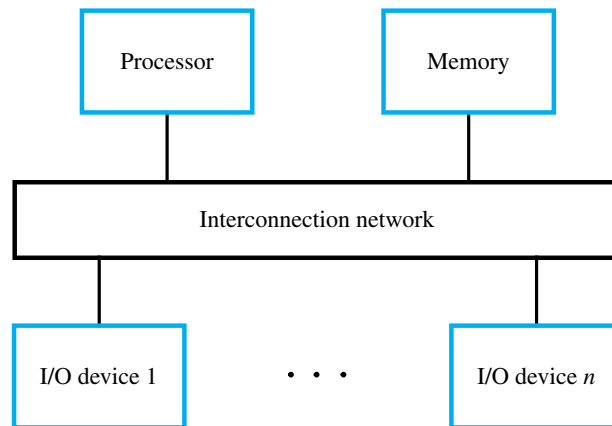


Figure 3.1 A computer system.

extended to deal with the I/O devices as well. For this purpose, each I/O device must appear to the processor as consisting of some addressable locations, just like the memory. Some addresses in the address space of the processor are assigned to these I/O locations, rather than to the main memory. These locations are usually implemented as bit storage circuits (flip-flops) organized in the form of registers. It is customary to refer to them as *I/O registers*. Since the I/O devices and the memory share the same address space, this arrangement is called *memory-mapped I/O*. It is used in most computers.

With memory-mapped I/O, any machine instruction that can access memory can be used to transfer data to or from an I/O device. For example, if DATAIN is the address of a register in an input device, the instruction

Load R2, DATAIN

reads the data from the DATAIN register and loads them into processor register R2. Similarly, the instruction

Store R2, DATAOUT

sends the contents of register R2 to location DATAOUT, which is a register in an output device.

3.1.1 I/O DEVICE INTERFACE

An I/O device is connected to the interconnection network by using a circuit, called the *device interface*, which provides the means for data transfer and for the exchange of status and control information needed to facilitate the data transfers and govern the operation of the device. The interface includes some registers that can be accessed by the processor. One register may serve as a buffer for data transfers, another may hold information about the current status of the device, and yet another may store the information that controls the operational behavior of the device. These *data*, *status*, and *control* registers are accessed by program instructions as if they were memory locations. Typical transfers of information are between I/O registers and the registers in the processor. Figure 3.2 illustrates how the keyboard and display devices are connected to the processor from the software point of view.

3.1.2 PROGRAM-CONTROLLED I/O

Let us begin the discussion of input/output issues by looking at two essential I/O devices for human-computer interaction—keyboard and display. Consider a task that reads characters typed on a keyboard, stores these data in the memory, and displays the same characters on a display screen. A simple way of implementing this task is to write a program that performs all functions needed to realize the desired action. This method is known as *program-controlled I/O*.

In addition to transferring each character from the keyboard into the memory, and then to the display, it is necessary to ensure that this happens at the right time. An input character must be read in response to a key being pressed. For output, a character must be sent to

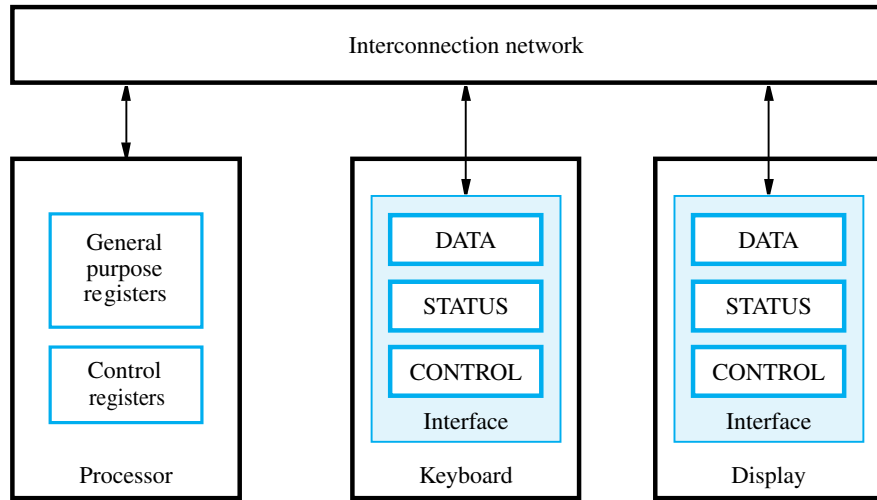


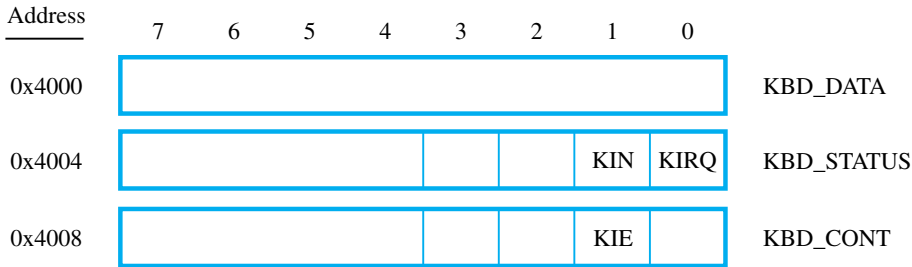
Figure 3.2 The connection for processor, keyboard, and display.

the display only when the display device is able to accept it. The rate of data transfer from the keyboard to a computer is limited by the typing speed of the user, which is unlikely to exceed a few characters per second. The rate of output transfers from the computer to the display is much higher. It is determined by the rate at which characters can be transmitted to and displayed on the display device, typically several thousand characters per second. However, this is still much slower than the speed of a processor that can execute billions of instructions per second. The difference in speed between the processor and I/O devices creates the need for mechanisms to synchronize the transfer of data between them.

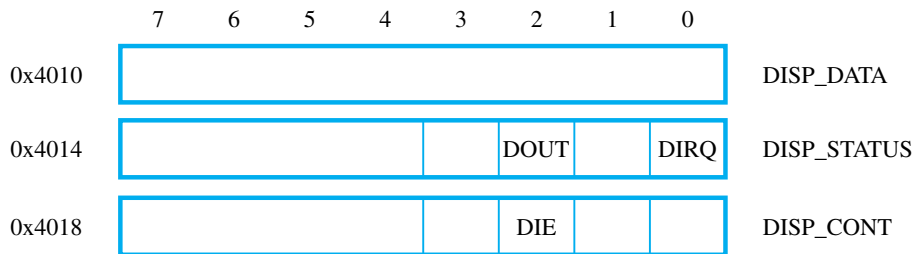
One solution to this problem involves a signaling protocol. On output, the processor sends the first character and then waits for a signal from the display that the next character can be sent. It then sends the second character, and so on. An input character is obtained from the keyboard in a similar way. The processor waits for a signal indicating that a key has been pressed and that a binary code that represents the corresponding character is available in an I/O register associated with the keyboard. Then the processor proceeds to read that code.

The keyboard includes a circuit that responds to a key being pressed by producing the code for the corresponding character that can be used by the computer. We will assume that ASCII code (presented in Table 1.1) is used, in which each character code occupies one byte. Let `KBD_DATA` be the address label of an 8-bit register that holds the generated character. Also, let a signal indicating that a key has been pressed be provided by setting to 1 a flip-flop called `KIN`, which is a part of an eight-bit status register, `KBD_STATUS`. The processor can read the *status flag* `KIN` to determine when a character code has been placed in `KBD_DATA`. When the processor reads the status flag to determine its state, we say that the processor *polls* the I/O device.

The display includes an 8-bit register, which we will call `DISP_DATA`, used to receive characters from the processor. It also must be able to indicate that it is ready to receive the



(a) Keyboard interface



(b) Display interface

Figure 3.3 Registers in the keyboard and display interfaces.

next character; this can be done by using a status flag called DOUT, which is one bit in a status register, DISP_STATUS.

Figure 3.3 illustrates how these registers may be organized. The interface for each device also includes a control register, which we will discuss in Section 3.2. We have identified only a few bits in the registers, those that are pertinent to the discussion in this chapter. Other bits can be used for other purposes, or perhaps simply ignored.

If the registers in I/O interfaces are to be accessed as if they are memory locations, each register must be assigned a specific address that will be recognized by the interface circuit. In Figure 3.3, we assigned hexadecimal numbers 4000 and 4010 as base addresses for the keyboard and display, respectively. These are the addresses of the data registers. The addresses of the status registers are four bytes higher, and the control registers are eight bytes higher. This makes all addresses word-aligned in a 32-bit word computer, which is usually done in practice. Assigning the addresses to registers in this manner makes the I/O registers accessible in a program executed by the processor. This is the programmer's view of the device.

A program is needed to perform the task of reading the characters produced by the keyboard, storing these characters in the memory, and sending them to the display. To perform I/O transfers, the processor must execute machine instructions that check the state of the status flags and transfer data between the processor and the I/O devices.

Let us consider the details of the input process. When a key is pressed, the keyboard circuit places the ASCII-encoded character into the KBD_DATA register. At the same time, the circuit sets the KIN flag to 1. Meanwhile, the processor is executing the I/O program which continuously checks the state of the KIN flag. When it detects that KIN is set to 1, it transfers the contents of KBD_DATA into a processor register. Once the contents of KBD_DATA are read, KIN must be cleared to 0, which is usually done automatically by the interface circuit. If a second character is entered at the keyboard, KIN is again set to 1 and the process repeats. The desired action can be achieved by performing the operations:

```

READWAIT      Read the KIN flag
               Branch to READWAIT if KIN = 0
               Transfer data from KBD_DATA to R5

```

which reads the character into processor register R5.

An analogous process takes place when characters are transferred from the processor to the display. When DOUT is equal to 1, the display is ready to receive a character. Under program control, the processor monitors DOUT, and when DOUT is equal to 1, the processor transfers an ASCII-encoded character to DISP_DATA. The transfer of a character to DISP_DATA clears DOUT to 0. When the display device is ready to receive a second character, DOUT is again set to 1. This can be achieved by performing the operations:

```

WRITEWAIT     Read the DOUT flag
               Branch to WRITEWAIT if DOUT = 0
               Transfer data from R5 to DISP_DATA

```

The wait loop is executed repeatedly until the status flag DOUT is set to 1 by the display when it is free to receive a character. Then, the character from R5 is transferred to DISP_DATA to be displayed, which also clears DOUT to 0.

We assume that the initial state of KIN is 0 and the initial state of DOUT is 1. This initialization is normally performed by the device control circuits when power is turned on.

In computers that use memory-mapped I/O, in which some addresses are used to refer to registers in I/O interfaces, data can be transferred between these registers and the processor using instructions such as Load, Store, and Move. For example, the contents of the keyboard character buffer KBD_DATA can be transferred to register R5 in the processor by the instruction

```
LoadByte  R5, KBD_DATA
```

Similarly, the contents of register R5 can be transferred to DISP_DATA by the instruction

```
StoreByte R5, DISP_DATA
```

The LoadByte and StoreByte operation codes signify that the operand size is a byte, to distinguish them from the Load and Store operation codes that we have used for word operands.

The Read operation described above may be implemented by the RISC-style instructions:

READWAIT:	LoadByte	R4, KBD_STATUS
	And	R4, R4, #2
	Branch_if_[R4]=0	READWAIT
	LoadByte	R5, KBD_DATA

The And instruction is used to test the KIN flag, which is bit b_1 of the status information in R4 that was read from the KBD_STATUS register. As long as $b_1 = 0$, the result of the AND operation leaves the value in R4 equal to zero, and the READWAIT loop continues to be executed.

Similarly, the Write operation may be implemented as:

WRITEWAIT:	LoadByte	R4, DISP_STATUS
	And	R4, R4, #4
	Branch_if_[R4]=0	WRITEWAIT
	StoreByte	R5, DISP_DATA

Observe that the And instruction in this case uses the immediate value 4 to test the display's status bit, b_2 .

3.1.3 AN EXAMPLE OF A RISC-STYLE I/O PROGRAM

We can now put together a complete program for a typical I/O task, as shown in Figure 3.4. The program uses the program-controlled I/O approach described above to read, store, and display a line of characters typed at the keyboard. As the characters are read in, one by one, they are stored in the memory and then *echoed* back to the display. The program finishes when the carriage return character, CR, is encountered. The address of the first byte location of the memory where the line is to be stored is LOC. Register R2 is used to point to this part of the memory, and it is initially loaded with the address LOC by the first instruction in the program. R2 is incremented for each character read and displayed.

3.1.4 AN EXAMPLE OF A CISC-STYLE I/O PROGRAM

Let us now perform the same task using CISC-style instructions. In CISC instruction sets it is possible to perform some arithmetic and logic operations directly on operands in the memory. So, it is possible to have the instruction

TestBit destination, #k

which tests bit b_k of the destination operand and sets the condition flag Z (Zero) to 1 if $b_k = 0$ and to 0 otherwise. Since the operand can be in a memory location, we can use the instruction

TestBit KBD_STATUS, #1

	Move	R2, #LOC	Initialize pointer register R2 to point to the address of the first location in main memory where the characters are to be stored.
READ:	MoveByte	R3, #CR	Load ASCII code for Carriage Return into R3.
	LoadByte	R4, KBD_STATUS	Wait for a character to be entered.
	And	R4, R4, #2	Check the KIN flag.
	Branch_if_[R4]=0	READ	
	LoadByte	R5, KBD_DATA	Read the character from KBD_DATA (this clears KIN to 0).
	StoreByte	R5, (R2)	Write the character into the main memory and
	Add	R2, R2, #1	increment the pointer to main memory.
ECHO:	LoadByte	R4, DISP_STATUS	Wait for the display to become ready.
	And	R4, R4, #4	Check the DOUT flag.
	Branch_if_[R4]=0	ECHO	
	StoreByte	R5, DISP_DATA	Move the character just read to the display buffer register (this clears DOUT to 0).
	Branch_if_[R5]≠[R3]	READ	Check if the character just read is the Carriage Return. If it is not, then branch back and read another character.

Figure 3.4 A RISC-style program that reads a line of characters and displays it.

to test the state of the KIN flag in the keyboard interface. A Branch instruction that checks the state of the Z flag can then be used to cause a branch to the beginning of the wait loop.

Figure 3.5 gives a CISC-style program that reads and displays a line of characters. Observe that the first MoveByte instruction transfers each character directly from KBD_DATA to the memory location pointed to by R2. A Compare instruction

Compare destination, source

performs the comparison by subtracting the contents of the source from the contents of the destination, and then sets the condition flags based on the result. It does not change the contents of either the source or the destination. Note that the CompareByte instruction in Figure 3.5 uses the autoincrement addressing mode, which automatically increments the value of the pointer R2 after the comparison has been made. In the RISC-style program in Figure 3.4 the pointer has to be incremented using a separate Add instruction.

We have discussed the memory-mapped I/O scheme, which is used in most computers. There is an alternative that can be found in some processors where there exist special In and Out instructions to perform I/O transfers. In this case, there exists a separate I/O address space used only by these instructions. When building a computer system that uses these processors, the designer has the option of connecting I/O devices to use the special I/O address space or simply incorporating them as part of the memory address space.

	Move	R2, #LOC	Initialize pointer register R2 to point to the address of the first location in main memory where the characters are to be stored.
READ:	TestBit	KBD_STATUS, #1	Wait for a character to be entered in the keyboard buffer KBD_DATA.
	Branch=0	READ	
	MoveByte	(R2), KBD_DATA	Transfer the character from KBD_DATA into the main memory (this clears KIN to 0).
ECHO:	TestBit	DISP_STATUS, #2	Wait for the display to become ready.
	Branch=0	ECHO	
	MoveByte	DISP_DATA, (R2)	Move the character just read to the display buffer register (this clears DOUT to 0).
	CompareByte	(R2)+, #CR	Check if the character just read is CR (carriage return). If it is not CR, then branch back and read another character.
	Branch≠0	READ	Also, increment the pointer to store the next character.

Figure 3.5 A CISC-style program that reads a line of characters and displays it.

Program-controlled I/O requires continuous involvement of the processor in the I/O activities. Almost all of the execution time for the programs in Figures 3.4 and 3.5 is spent in the two wait loops, while the processor waits for a key to be pressed or for the display to become available. Wasting the processor execution time in this manner can be avoided by using the concept of interrupts.

3.2 INTERRUPTS

In the examples in Figures 3.4 and 3.5, the program enters a wait loop in which it repeatedly tests the device status. During this period, the processor is not performing any useful computation. There are many situations where other tasks can be performed while waiting for an I/O device to become ready. To allow this to happen, we can arrange for the I/O device to alert the processor when it becomes ready. It can do so by sending a hardware signal called an *interrupt request* to the processor. Since the processor is no longer required to continuously poll the status of I/O devices, it can use the waiting period to perform other useful tasks. Indeed, by using interrupts, such waiting periods can ideally be eliminated.

Consider a task that requires continuous extensive computations to be performed and the results to be displayed on a display device. The displayed results must be updated every ten seconds. The ten-second intervals can be determined by a simple timer circuit, which

Example 3.1

generates an appropriate signal. The processor treats the timer circuit as an input device that produces a signal that can be interrogated. If this is done by means of polling, the processor will waste considerable time checking the state of the signal. A better solution is to have the timer circuit raise an interrupt request once every ten seconds. In response, the processor displays the latest results.

The task can be implemented with a program that consists of two routines, COMPUTE and DISPLAY. The processor continuously executes the COMPUTE routine. When it receives an interrupt request from the timer, it suspends the execution of the COMPUTE routine and executes the DISPLAY routine which sends the latest results to the display device. Upon completion of the DISPLAY routine, the processor resumes the execution of the COMPUTE routine. Since the time needed to send the results to the display device is very small compared to the ten-second interval, the processor in effect spends almost all of its time executing the COMPUTE routine.

This example illustrates the concept of interrupts. The routine executed in response to an interrupt request is called the *interrupt-service routine*, which is the DISPLAY routine in our example. Interrupts bear considerable resemblance to subroutine calls. Assume that an interrupt request arrives during execution of instruction i in Figure 3.6. The processor first completes execution of instruction i . Then, it loads the program counter with the address of the first instruction of the interrupt-service routine. For the time being, let us assume that this address is hardwired in the processor. After execution of the interrupt-service routine, the processor returns to instruction $i + 1$. Therefore, when an interrupt occurs, the current contents of the PC, which point to instruction $i + 1$, must be put in temporary storage in a known location. A Return-from-interrupt instruction at the end of the interrupt-service routine reloads the PC from that temporary storage location, causing execution to resume at

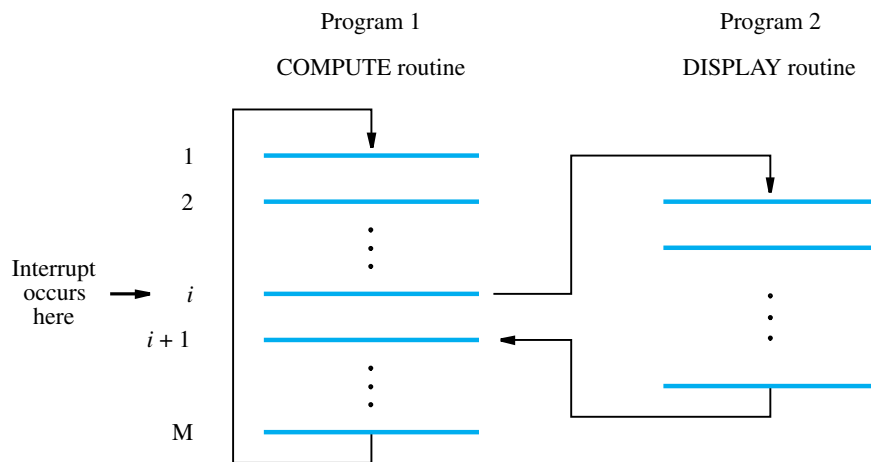


Figure 3.6 Transfer of control through the use of interrupts.

instruction $i + 1$. The *return address* must be saved either in a designated general-purpose register or on the processor stack.

We should note that as part of handling interrupts, the processor must inform the device that its request has been recognized so that it may remove its interrupt-request signal. This can be accomplished by means of a special control signal, called *interrupt acknowledge*, which is sent to the device through the interconnection network. An alternative is to have the transfer of data between the processor and the I/O device interface accomplish the same purpose. The execution of an instruction in the interrupt-service routine that accesses the status or data register in the device interface implicitly informs the device that its interrupt request has been recognized.

So far, treatment of an interrupt-service routine is very similar to that of a subroutine. An important departure from this similarity should be noted. A subroutine performs a function required by the program from which it is called. As such, potential changes to status information and contents of registers are anticipated. However, an interrupt-service routine may not have any relation to the portion of the program being executed at the time the interrupt request is received. Therefore, before starting execution of the interrupt-service routine, status information and contents of processor registers that may be altered in unanticipated ways during the execution of that routine must be saved. This saved information must be restored before execution of the interrupted program is resumed. In this way, the original program can continue execution without being affected in any way by the interruption, except for the time delay.

The task of saving and restoring information can be done automatically by the processor or by program instructions. Most modern processors save only the minimum amount of information needed to maintain the integrity of program execution. This is because the process of saving and restoring registers involves memory transfers that increase the total execution time, and hence represent execution overhead. Saving registers also increases the delay between the time an interrupt request is received and the start of execution of the interrupt-service routine. This delay is called *interrupt latency*. In some applications, a long interrupt latency is unacceptable. For these reasons, the amount of information saved automatically by the processor when an interrupt request is accepted should be kept to a minimum. Typically, the processor saves only the contents of the program counter and the processor status register. Any additional information that needs to be saved must be saved by explicit instructions at the beginning of the interrupt-service routine and restored at the end of the routine. In some earlier processors, particularly those with a small number of registers, all registers are saved automatically by the processor hardware at the time an interrupt request is accepted. The data saved are restored to their respective registers as part of the execution of the Return-from-interrupt instruction.

Some computers provide two types of interrupts. One saves all register contents, and the other does not. A particular I/O device may use either type, depending upon its response-time requirements. Another interesting approach is to provide duplicate sets of processor registers. In this case, a different set of registers can be used by the interrupt-service routine, thus eliminating the need to save and restore registers. The duplicate registers are sometimes called the *shadow registers*.

An interrupt is more than a simple mechanism for coordinating I/O transfers. In a general sense, interrupts enable transfer of control from one program to another to be

initiated by an event external to the computer. Execution of the interrupted program resumes after the execution of the interrupt-service routine has been completed. The concept of interrupts is used in operating systems and in many control applications where processing of certain routines must be accurately timed relative to external events. The latter type of application is referred to as *real-time processing*.

3.2.1 ENABLING AND DISABLING INTERRUPTS

The facilities provided in a computer must give the programmer complete control over the events that take place during program execution. The arrival of an interrupt request from an external device causes the processor to suspend the execution of one program and start the execution of another. Because interrupts can arrive at any time, they may alter the sequence of events from that envisaged by the programmer. Hence, the interruption of program execution must be carefully controlled. A fundamental facility found in all computers is the ability to enable and disable such interruptions as desired.

There are many situations in which the processor should ignore interrupt requests. For instance, the timer circuit in Example 3.1 should raise interrupt requests only when the COMPUTE routine is being executed. It should be prevented from doing so when some other task is being performed. In another case, it may be necessary to guarantee that a particular sequence of instructions is executed to the end without interruption because the interrupt-service routine may change some of the data used by the instructions in question. For these reasons, some means for enabling and disabling interrupts must be available to the programmer.

It is convenient to be able to enable and disable interrupts at both the processor and I/O device ends. The processor can either accept or ignore interrupt requests. An I/O device can either be allowed to raise interrupt requests or prevented from doing so. A commonly used mechanism to achieve this is to use some control bits in registers that can be accessed by program instructions.

The processor has a *status register* (PS), which contains information about its current state of operation. Let one bit, IE, of this register be assigned for enabling/disabling interrupts. Then, the programmer can set or clear IE to cause the desired action. When $IE = 1$, interrupt requests from I/O devices are accepted and serviced by the processor. When $IE = 0$, the processor simply ignores all interrupt requests from I/O devices.

The interface of an I/O device includes a control register that contains the information that governs the mode of operation of the device. One bit in this register may be dedicated to interrupt control. The I/O device is allowed to raise interrupt requests only when this bit is set to 1. We will discuss this arrangement in Section 3.2.3.

Let us now consider the specific case of a single interrupt request from one device. When a device activates the interrupt-request signal, it keeps this signal activated until it learns that the processor has accepted its request. This means that the interrupt-request signal will be active during execution of the interrupt-service routine, perhaps until an instruction is reached that accesses the device in question. It is essential to ensure that this active request signal does not lead to successive interruptions, causing the system to enter an infinite loop from which it cannot recover.

A good choice is to have the processor automatically disable interrupts before starting the execution of the interrupt-service routine. The processor saves the contents of the program counter and the processor status register. After saving the contents of the PS register, with the IE bit equal to 1, the processor clears the IE bit in the PS register, thus disabling further interrupts. Then, it begins execution of the interrupt-service routine. When a Return-from-interrupt instruction is executed, the saved contents of the PS register are restored, setting the IE bit back to 1. Hence, interrupts are again enabled.

Before proceeding to study more complex aspects of interrupts, let us summarize the sequence of events involved in handling an interrupt request from a single device. Assuming that interrupts are enabled in both the processor and the device, the following is a typical scenario:

1. The device raises an interrupt request.
2. The processor interrupts the program currently being executed and saves the contents of the PC and PS registers.
3. Interrupts are disabled by clearing the IE bit in the PS to 0.
4. The action requested by the interrupt is performed by the interrupt-service routine, during which time the device is informed that its request has been recognized, and in response, it deactivates the interrupt-request signal.
5. Upon completion of the interrupt-service routine, the saved contents of the PC and PS registers are restored (enabling interrupts by setting the IE bit to 1), and execution of the interrupted program is resumed.

3.2.2 HANDLING MULTIPLE DEVICES

Let us now consider the situation where a number of devices capable of initiating interrupts are connected to the processor. Because these devices are operationally independent, there is no definite order in which they will generate interrupts. For example, device X may request an interrupt while an interrupt caused by device Y is being serviced, or several devices may request interrupts at exactly the same time. This gives rise to a number of questions:

1. How can the processor determine which device is requesting an interrupt?
2. Given that different devices are likely to require different interrupt-service routines, how can the processor obtain the starting address of the appropriate routine in each case?
3. Should a device be allowed to interrupt the processor while another interrupt is being serviced?
4. How should two or more simultaneous interrupt requests be handled?

The means by which these issues are handled vary from one computer to another, and the approach taken is an important consideration in determining the computer's suitability for a given application.

When an interrupt request is received it is necessary to identify the particular device that raised the request. Furthermore, if two devices raise interrupt requests at the same time,

it must be possible to break the tie and select one of the two requests for service. When the interrupt-service routine for the selected device has been completed, the second request can be serviced.

The information needed to determine whether a device is requesting an interrupt is available in its status register. When the device raises an interrupt request, it sets to 1 a bit in its status register, which we will call the IRQ bit. The simplest way to identify the interrupting device is to have the interrupt-service routine poll all I/O devices in the system. The first device encountered with its IRQ bit set to 1 is the device that should be serviced. An appropriate subroutine is then called to provide the requested service.

The polling scheme is easy to implement. Its main disadvantage is the time spent interrogating the IRQ bits of devices that may not be requesting any service. An alternative approach is to use vectored interrupts, which we describe next.

Vectored Interrupts

To reduce the time involved in the polling process, a device requesting an interrupt may identify itself directly to the processor. Then, the processor can immediately start executing the corresponding interrupt-service routine. The term *vectored interrupts* refers to interrupt-handling schemes based on this approach.

A device requesting an interrupt can identify itself if it has its own interrupt-request signal, or if it can send a special code to the processor through the interconnection network. The processor's circuits determine the memory address of the required interrupt-service routine. A commonly used scheme is to allocate permanently an area in the memory to hold the addresses of interrupt-service routines. These addresses are usually referred to as *interrupt vectors*, and they are said to constitute the *interrupt-vector table*. For example, 128 bytes may be allocated to hold a table of 32 interrupt vectors. Typically, the interrupt-vector table is in the lowest-address range. The interrupt-service routines may be located anywhere in the memory. When an interrupt request arrives, the information provided by the requesting device is used as a pointer into the interrupt-vector table, and the address in the corresponding interrupt vector is automatically loaded into the program counter.

Interrupt Nesting

We suggested in Section 3.2.1 that interrupts should be disabled during the execution of an interrupt-service routine, to ensure that a request from one device will not cause more than one interruption. The same arrangement is often used when several devices are involved, in which case execution of a given interrupt-service routine, once started, always continues to completion before the processor accepts an interrupt request from a second device. Interrupt-service routines are typically short, and the delay they may cause is acceptable for most simple devices.

For some devices, however, a long delay in responding to an interrupt request may lead to erroneous operation. Consider, for example, a computer that keeps track of the time of day using a real-time clock. This is a device that sends interrupt requests to the processor at regular intervals. For each of these requests, the processor executes a short interrupt-service routine to increment a set of counters in the memory that keep track of time in seconds, minutes, and so on. Proper operation requires that the delay in responding to an interrupt request from the real-time clock be small in comparison with the interval between

two successive requests. To ensure that this requirement is satisfied in the presence of other interrupting devices, it may be necessary to accept an interrupt request from the clock during the execution of an interrupt-service routine for another device, i.e., to nest interrupts.

This example suggests that I/O devices should be organized in a priority structure. An interrupt request from a high-priority device should be accepted while the processor is servicing a request from a lower-priority device.

A multiple-level priority organization means that during execution of an interrupt-service routine, interrupt requests will be accepted from some devices but not from others, depending upon the device's priority. To implement this scheme, we can assign a priority level to the processor that can be changed under program control. The priority level of the processor is the priority of the program that is currently being executed. The processor accepts interrupts only from devices that have priorities higher than its own. At the time that execution of an interrupt-service routine for some device is started, the priority of the processor is raised to that of the device either automatically or with special instructions. This action disables interrupts from devices that have the same or lower level of priority. However, interrupt requests from higher-priority devices will continue to be accepted. The processor's priority can be encoded in a few bits of the processor status register. While this scheme is used in some processors, we will use a simpler scheme in later examples.

Finally, we should point out that if nested interrupts are allowed, then each interrupt-service routine must save on the stack the saved contents of the program counter and the status register. This has to be done before the interrupt-service routine enables nesting by setting the IE bit in the status register to 1.

Simultaneous Requests

We also need to consider the problem of simultaneous arrivals of interrupt requests from two or more devices. The processor must have some means of deciding which request to service first. Polling the status registers of the I/O devices is the simplest such mechanism. In this case, priority is determined by the order in which the devices are polled. When vectored interrupts are used, we must ensure that only one device is selected to send its interrupt vector code. This is done in hardware, by using arbitration circuits which we will discuss in Chapter 7.

3.2.3 CONTROLLING I/O DEVICE BEHAVIOR

It is important to ensure that interrupt requests are generated only by those I/O devices that the processor is currently willing to recognize. Hence, we need a mechanism in the interface circuits of individual devices to control whether a device is allowed to interrupt the processor. The control needed is usually provided in the form of an *interrupt-enable* bit in the device's interface circuit.

I/O devices vary in complexity from simple to quite complex. Simple devices, such as a keyboard, require little in the way of control. Complex devices may have a number of possible modes of operation, which must be controlled. A commonly used approach is to provide a control register in the device interface, which holds the information needed to control the behavior of the device. This register is accessed as an addressable location, just

like the data and status registers that we discussed before. One bit in the register serves as the interrupt-enable bit, IE. When it is set to 1 by an instruction that writes new information into the control register, the device is placed into a mode in which it is allowed to interrupt the processor whenever it is ready for an I/O transfer.

Figure 3.3 shows the registers that may be used in the interfaces of keyboard and display devices. Since these devices transfer character-based data, handling one character at a time, it is appropriate to use an eight-bit data register. We have assumed that the status and control registers are also eight bits long. Only one or two bits in these registers are needed in handling the I/O transfers. The remaining bits can be used to specify other aspects of the operation of the device, or ignored if they are not needed. The keyboard status register includes bits KIN and KIRQ. We have already discussed the use of the KIN bit in Section 3.1.2. The KIRQ bit is set to 1 if an interrupt request has been raised, but not yet serviced. The keyboard may raise interrupt requests only when the interrupt-enable bit, KIE, in its control register is set to 1. Thus, when both KIE and KIN bits are equal to 1, an interrupt request is raised and the KIRQ bit is set to 1. Similarly, the DIRQ bit in the status register of the display interface indicates whether an interrupt request has been raised. Bit DIE in the control register of this interface is used to enable interrupts. Observe that we have placed KIN and KIE in bit position 1, and DOUT and DIE in position 2. This is an arbitrary choice that makes the program examples that follow easier to understand.

3.2.4 PROCESSOR CONTROL REGISTERS

We have already discussed the need for a status register in the processor. To deal with interrupts it is useful to have some other control registers. Figure 3.7 depicts one possibility, where there are four processor control registers. The status register, PS, includes the interrupt-enable bit, IE, in addition to other status information. Recall that the processor will accept interrupts only when this bit is set to 1. The IPS register is used to automatically

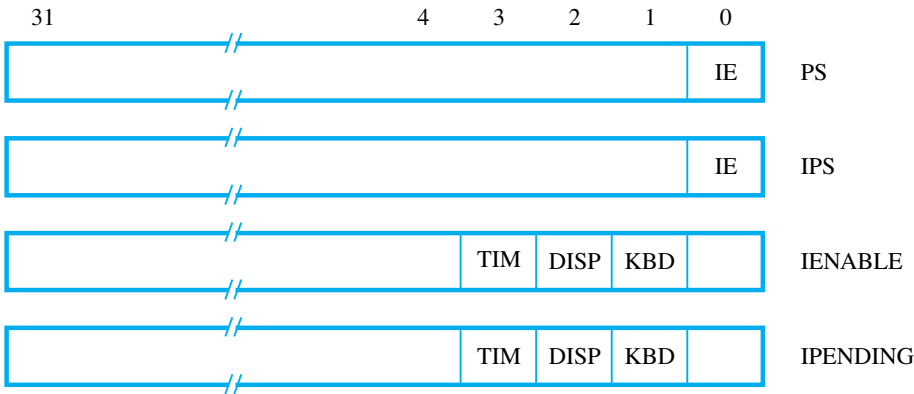


Figure 3.7 Control registers in the processor.

save the contents of PS when an interrupt request is received and accepted. At the end of the interrupt-service routine, the previous state of the processor is automatically restored by transferring the contents of IPS into PS. Since there is only one register available for storing the previous status information, it becomes necessary to save the contents of IPS on the stack if nested interrupts are allowed.

The IENABLE register allows the processor to selectively respond to individual I/O devices. A bit may be assigned for each device, as shown in the figure for the keyboard, display, and a timer circuit that we will use in a later example. When a bit is set to 1, the processor will accept interrupt requests from the corresponding device. The IPENDING register indicates the active interrupt requests. This is convenient when multiple devices may raise requests at the same time. Then, a program can decide which interrupt should be serviced first.

In a 32-bit processor, the control registers are 32 bits long. Using the structure in Figure 3.7, it is possible to accommodate 32 I/O devices in a straightforward manner.

Assembly-language instructions can refer to processor control registers by using names such as those in Figure 3.7. But, these registers cannot be accessed in the same way as the general-purpose registers. They cannot be accessed by arithmetic and logic instructions. They also cannot be accessed by Load and Store instructions that use the encoding format depicted in Figure 2.32c, because a five-bit field is used to specify a source or a destination register in these instructions, which makes it possible to specify only 32 general-purpose registers. Special instructions or special addressing modes may be provided to access the processor control registers. In a RISC-style processor, the special instructions may be of the type

MoveControl R2, PS

which loads the contents of the program status register into register R2, and

MoveControl IENABLE, R3

which places the contents of R3 into the IENABLE register. These instructions perform transfers between control and general-purpose registers.

3.2.5 EXAMPLES OF INTERRUPT PROGRAMS

Having presented the basic aspects of interrupts, we can now give some illustrative examples. We will use the keyboard and display devices with the register structure given in Figure 3.3.

Let us consider again the task of reading a line of characters typed on a keyboard, storing the characters in the main memory, and displaying them on a display device. In Figures 3.4 and 3.5, we showed how this task may be performed by using the polling approach to detect when the I/O devices are ready for data transfer. Now, we will use interrupts with the keyboard, but polling with the display.

Example 3.2

We assume for now that a specific memory location, `ILOC`, is dedicated for dealing with interrupts, and that it contains the first instruction of the interrupt-service routine. Whenever an interrupt request arrives at the processor, and processor interrupts are enabled, the processor will automatically:

- Save the contents of the program counter, either in a processor register that holds the return address or on the processor stack.
- Save the contents of the status register `PS` by transferring them into the `IPS` register, and clear the `IE` bit in the `PS`.
- Load the address `ILOC` into the program counter.

Assume that in the Main program we wish to read a line from the keyboard and store the characters in successive byte locations in the memory, starting at location `LINE`. Also, assume that the interrupt-service routine has been loaded in the memory, starting at location `ILOC`. The Main program has to initialize the interrupt process as follows:

1. Load the address `LINE` into a memory location `PNTR`. The interrupt-service routine will use this location as a pointer to store the input characters in the memory.
2. Enable interrupts in the keyboard interface by setting to 1 the `KIE` bit in the `KBD_CONT` register.
3. Enable the processor to accept interrupts from the keyboard by setting to 1 the `KBD` bit in its control register `IENABLE`.
4. Enable the processor to respond to interrupts in general by setting to 1 the `IE` bit in the processor status register, `PS`.

Once this initialization is completed, typing a character on the keyboard will cause an interrupt request to be generated by the keyboard interface. The program being executed at that time will be interrupted and the interrupt-service routine will be executed. This routine must perform the following tasks:

1. Read the input character from the keyboard input data register. This will cause the interface circuit to remove its interrupt request.
2. Store the character in the memory location pointed to by `PNTR`, and increment `PNTR`.
3. Display the character using the polling approach.
4. When the end of the line is reached, disable keyboard interrupts and inform the Main program.
5. Return from interrupt.

A RISC-style program that performs these tasks is shown in Figure 3.8. The comments in the program explain the relevant details. When the end of the input line is detected, the interrupt-service routine clears the `KIE` bit in register `KBD_CONT`, as no further input is expected. It also sets to 1 the variable `EOL` (End Of Line), which was initially cleared to 0. We assume that it is checked periodically by the Main program to determine when the input line is ready for processing. The `EOL` variable provides a means of signaling between the Main program and the interrupt-service routine.

Interrupt-service routine

ILOC:	Subtract	SP, SP, #8	Save registers.
	Store	R2, 4(SP)	
	Store	R3, (SP)	
	Load	R2, PNTR	Load address pointer.
	LoadByte	R3, KBD_DATA	Read character from keyboard.
	StoreByte	R3, (R2)	Write the character into memory
	Add	R2, R2, #1	and increment the pointer.
	Store	R2, PNTR	Update the pointer in memory.
ECHO:	LoadByte	R2, DISP_STATUS	Wait for display to become ready.
	And	R2, R2, #4	
	Branch_if_[R2]=0	ECHO	
	StoreByte	R3, DISP_DATA	Display the character just read.
	Move	R2, #CR	ASCII code for Carriage Return.
	Branch_if_[R3]≠[R2]	RTRN	Return if not CR.
	Move	R2, #1	
	Store	R2, EOL	Indicate end of line.
	Clear	R2	Disable interrupts in
	StoreByte	R2, KBD_CONT	the keyboard interface.
RTRN:	Load	R3, (SP)	Restore registers.
	Load	R2, 4(SP)	
	Add	SP, SP, #8	
	Return-from-interrupt		

Main program

START:	Move	R2, #LINE	
	Store	R2, PNTR	Initialize buffer pointer.
	Clear	R2	
	Store	R2, EOL	Clear end-of-line indicator.
	Move	R2, #2	Enable interrupts in
	StoreByte	R2, KBD_CONT	the keyboard interface.
	MoveControl	R2, IENABLE	
	Or	R2, R2, #2	Enable keyboard interrupts in
	MoveControl	IENABLE, R2	the processor control register.
	MoveControl	R2, PS	
	Or	R2, R2, #1	
	MoveControl	PS, R2	Set interrupt-enable bit in PS.
	next instruction		

Figure 3.8 A RISC-style program that reads a line of characters using interrupts, and displays the line using polling.

Observe that the last three instructions in the Main program are used to set to 1 the interrupt-enable bit in PS. Since only MoveControl instructions can access the contents of a control register, the contents of PS are loaded into a general-purpose register, R2, modified and then written back into PS. Using the Or instruction to modify the contents affects only the IE bit and leaves the rest of the bits in PS unchanged.

When multiple I/O devices raise interrupt requests, it is necessary to determine which device has requested an interrupt. This can be done in software by checking the information in the IPENDING control register and choosing the interrupt-service routine that should be executed.

Example 3.3 In Example 3.2, we used interrupts with the keyboard only. The display device can also use interrupts. Suppose a program needs to display a page of text stored in the memory. This can be done by having the processor send a character whenever the display interface is ready, which may be indicated by an interrupt request. Assume that both the display and the keyboard are used by this program, and that both are enabled to raise interrupt requests. Using the register structure in Figures 3.3 and 3.7, the initialization of interrupts and the processing of requests can be done as indicated in Figure 3.9.

The Main program must initialize any variables needed by the interrupt-service routines, such as the memory buffer pointers. Then, it enables interrupts in both the keyboard and display interfaces. Next, it enables interrupts in the processor control register IENABLE. Note that the immediate value 6, which is loaded into this register, sets bits KBD and DISP to 1. Finally, the processor is enabled to respond to interrupts in general by setting to 1 the IE bit in the processor status register, PS.

Again, we assume that whenever an interrupt request arrives, the processor will automatically save the contents of the program counter (PC) and then load the address ILOC into PC. It will also save the contents of the status register (PS) by transferring them into the IPS register, and disable interrupts. Unlike Example 3.2, where we assumed that there is only one device that can raise interrupt requests, now we cannot go directly to the desired interrupt-service routine. First, it is necessary to identify the interrupting device. The needed information is found in the processor control register IPENDING. Since the interrupt-service routine uses registers R2 and R3 in this process, the contents of these registers must be saved on the stack and later restored. It is also necessary to save the contents of the subroutine linkage register, LINK_reg, because an interrupt can occur while some subroutine is being executed and the interrupt-service routine calls a subroutine. The circuit that detects interrupts sets to 1 the appropriate bit in IPENDING for each pending request. In Figure 3.9, the contents of IPENDING are loaded into general purpose register R2, and then examined to determine which interrupts are pending. If the display has a pending interrupt, then its interrupt-service routine is executed. If not, then a check is made for the keyboard. This may be followed by checking any other devices that could have pending requests. The order in which the bits in IPENDING are checked establishes a priority for the interrupting devices in case of simultaneous requests.

Interrupt handler

ILOC:	Subtract	SP, SP, #12	Save registers.
	Store	LINK_reg, 8(SP)	
	Store	R2, 4(SP)	
	Store	R3, (SP)	
	MoveControl	R2, IPENDING	Check contents of IPENDING.
	And	R3, R2, #4	Check if display raised the request.
	Branch_if_[R3]=0	TESTKBD	If not, check if keyboard.
	Call	DISR	Call the display ISR.
TESTKBD:	And	R3, R2, #2	Check if keyboard raised the request.
	Branch_if_[R3]=0	NEXT	If not, then check next device.
	Call	KISR	Call the keyboard ISR.
NEXT:	...		Check for other interrupts.
	Load	R3, (SP)	Restore registers.
	Load	R2, 4(SP)	
	Load	LINK_reg, 8(SP)	
	Add	SP, SP, #12	
	Return-from-interrupt		

Main program

START:	...		Set up parameters for ISRs.
	Move	R2, #2	Enable interrupts in
	StoreByte	R2, KBD_CONT	the keyboard interface.
	Move	R2, #4	Enable interrupts in
	StoreByte	R2, DISP_CONT	the display interface.
	MoveControl	R2, IENABLE	
	Or	R2, R2, #6	Enable interrupts in
	MoveControl	IENABLE, R2	the processor control register.
	MoveControl	R2, PS	
	Or	R2, R2, #1	
	MoveControl	PS, R2	Set interrupt-enable bit in PS.
	next instruction		

Keyboard interrupt-service routine

KISR:	...
	:
	:
	Return

Display interrupt-service routine

DISR:	...
	:
	:
	Return

Figure 3.9 A RISC-style program that initializes and handles interrupts.

The program parts that handle interrupt requests and provide the corresponding service to the requesting devices are often referred to as the *interrupt handler*. Note that while the interrupt handler starts at the fixed address ILOC, the individual interrupt-service routines are just subroutines that can be placed anywhere in the memory.

In Figure 3.9, we used a software approach to determine the interrupting device. In processors that use vectored interrupts, the circuit that detects interrupt requests automatically loads a different address into the program counter for each interrupt that is assigned a specific location in the interrupt-vector table. A separate interrupt-service routine is executed to completion for each pending request, even if multiple interrupt requests are raised at the same time.

CISC-style Examples of Interrupts

The above tasks can be implemented using CISC-style instructions using the same basic approach. The main difference is that some operations, such as testing a bit in an I/O register, can be done directly. The tasks in Examples 3.2 and 3.3 can be realized using the programs in Figures 3.10 and 3.11, respectively. The TestBit instruction is used to test the status flags. The SetBit and ClearBit instructions are used to set an individual bit in an I/O register to 1 and 0, respectively. The comments in the programs provide explanations of how the desired tasks are realized.

Input/output operations in a computer system are usually much more involved than our simple examples suggest. As we will describe in Chapter 4, the operating system of the computer performs these operations on behalf of user programs. In Chapter 7, we will discuss in detail the hardware used in I/O operations.

3.2.6 EXCEPTIONS

An interrupt is an event that causes the execution of one program to be suspended and the execution of another program to begin. So far, we have dealt only with interrupts caused by events associated with I/O data transfers. However, the interrupt mechanism is used in a number of other situations.

The term *exception* is often used to refer to any event that causes an interruption. Hence, I/O interrupts are one example of an exception. We now describe a few other kinds of exceptions.

Recovery from Errors

Computers use a variety of techniques to ensure that all hardware components are operating properly. For example, many computers include an error-checking code in the main memory, which allows detection of errors in the stored data. If an error occurs, the control hardware detects it and informs the processor by raising an interrupt.

The processor may also interrupt a program if it detects an error or an unusual condition while executing the instructions of this program. For example, the OP-code field of an instruction may not correspond to any legal instruction, or an arithmetic instruction may attempt a division by zero.

When exception processing is initiated as a result of such errors, the processor proceeds in exactly the same manner as in the case of an I/O interrupt request. It suspends the program

Interrupt-service routine

ILOC:	Move	– (SP), R2	Save register.
	Move	R2, PNTR	Load address pointer.
ECHO:	MoveByte	(R2), KBD_DATA	Write the character into memory
	Add	PNTR, #1	and increment the pointer.
	TestBit	DISP_STATUS, #2	Wait for the display to become ready.
	Branch=0	ECHO	
	MoveByte	DISP_DATA, (R2)	Display the character just read.
	CompareByte	(R2), #CR	Check if the character just read is CR.
	Branch≠0	RTRN	Return if not CR.
	Move	EOL, #1	Indicate end of line.
	ClearBit	KBD_CONT, #1	Disable interrupts in keyboard interface.
	Move	R2, (SP)+	Restore register.
RTRN:	Return-from-interrupt		

Main program

START:	Move	PNTR, #LINE	Initialize buffer pointer.
	Clear	EOL	Clear end-of-line indicator.
	SetBit	KBD_CONT, #1	Enable interrupts in keyboard interface.
	Move	R2, #2	Enable keyboard interrupts in
	MoveControl	IENABLE, R2	the processor control register.
	MoveControl	R2, PS	
	Or	R2, #1	
	MoveControl	PS, R2	Set interrupt-enable bit in PS.
	next instruction		

Figure 3.10 A CISC-style program that reads a line of characters using interrupts, and displays the line using polling.

being executed and starts an exception-service routine, which takes appropriate action to recover from the error, if possible, or to inform the user about it. Recall that in the case of an I/O interrupt, we assumed that the processor completes execution of the instruction in progress before accepting the interrupt. However, when an interrupt is caused by an error associated with the current instruction, that instruction cannot usually be completed, and the processor begins exception processing immediately.

Debugging

Another important type of exception is used as an aid in debugging programs. System software usually includes a program called a *debugger*, which helps the programmer find errors in a program. The debugger uses exceptions to provide two important facilities: trace mode and breakpoints. These facilities are described in detail in Chapter 4.

Interrupt handler

ILOC:	Move	– (SP), R2	Save registers.
	Move	– (SP), LINK_reg	
	MoveControl	R2, IPENDING	Check contents of IPENDING.
	TestBit	R2, #2	Check if display raised the request.
	Branch=0	TESTKBD	If not, check if keyboard.
	Call	DISR	Call the display ISR.
TESTKBD:	TestBit	R2, #1	Check if keyboard raised the request.
	Branch=0	NEXT	If not, then check next device.
	Call	KISR	Call the keyboard ISR.
NEXT:	...		Check for other interrupts.
	Move	LINK_reg, (SP)+	Restore registers.
	Move	R2, (SP)+	
	Return-from-interrupt		

Main program

START:	...	Set up parameters for ISRs.
	SetBit	KBD_CONT, #1
	SetBit	DISP_CONT, #2
	MoveControl	R2, IENABLE
	Or	R2, #6
	MoveControl	IENABLE, R2
	MoveControl	R2, PS
	Or	R2, #1
	MoveControl	PS, R2
	next instruction	Set interrupt-enable bit in PS.

Keyboard interrupt-service routine

```

KISR:    ...
        :
        :
        Return

```

Display interrupt-service routine

```

DISR:    ...
        :
        :
        Return

```

Figure 3.11 A CISC-style program that initializes and handles interrupts.

Use of Exceptions in Operating Systems

The operating system (OS) software coordinates the activities within a computer. It uses exceptions to communicate with and control the execution of user programs. It uses hardware interrupts to perform I/O operations. This topic is discussed in Chapter 4.

3.3 CONCLUDING REMARKS

In this chapter, we discussed two basic approaches to I/O transfers. The simplest technique is programmed I/O, in which the processor performs all of the necessary functions under direct control of program instructions. The second approach is based on the use of interrupts; this mechanism makes it possible to interrupt the normal execution of programs in order to service higher-priority requests that require more urgent attention. Although all computers have a mechanism for dealing with such situations, the complexity and sophistication of interrupt-handling schemes vary from one computer to another.

We dealt with the I/O issues from the programmer's point of view. In Chapter 7 we will consider the hardware aspects and some commonly used I/O standards.

3.4 SOLVED PROBLEMS

This section presents some examples of problems that a student may be asked to solve, and shows how such problems can be solved.

Problem: Assume that a memory location `BINARY` contains a 32-bit pattern. It is desired to display these bits as eight hexadecimal digits on a display device that has the interface depicted in Figure 3.3. Write a program that accomplishes this task.

Example 3.4

Solution: First it is necessary to convert the 32-bit pattern into hex digits that are represented as ASCII-encoded characters. A simple way of doing the conversion is to use the *table-lookup* approach. A 16-entry table has to be constructed to provide the ASCII code for each possible hex digit. Then, for each four-bit segment of the pattern in `BINARY`, the corresponding character can be looked up in the table and stored in a block of memory bytes starting at location `HEX`. Finally, the eight characters starting at `HEX` are sent to the display.

Figures 3.12 and 3.13 give RISC- and CISC-style programs, respectively, for the required task. The comments describe the detailed actions taken.

LOOP:	Load	R2, BINARY	Load the binary number.
	Move	R3, #8	R3 is a digit counter that is set to 8.
	Move	R4, #HEX	R4 points to the hex digits.
	RotateL	R2, R2, #4	Rotate the high-order digit into low-order position.
	And	R5, R2, #0xF	Extract next digit.
	LoadByte	R6, TABLE(R5)	Get ASCII code for the digit and
	StoreByte	R6, (R4)	store it in HEX number location.
	Subtract	R3, R3, #1	Decrement the digit counter.
DISPLAY:	Add	R4, R4, #1	Increment the pointer to hex digits.
	Branch_if_[R3]>0	LOOP	Loop back if not the last digit.
	Move	R3, #8	
	Move	R4, #HEX	
DLOOP:	LoadByte	R5, DISP_STATUS	Wait for display to become ready.
	And	R5, R5, #4	Check the DOUT flag.
	Branch_if_[R5]=0	DLOOP	
	LoadByte	R6, (R4)	Get the next ASCII character
	StoreByte	R6, DISP_DATA	and send it to the display.
	Subtract	R3, R3, #1	Decrement the counter.
	Add	R4, R4, #1	Increment the character pointer.
	Branch_if_[R3]>0	DLOOP	Loop until all characters displayed.
next instruction			
HEX: TABLE:	ORIGIN	1000	
	RESERVE	8	Space for ASCII-encoded digits.
	DATABYTE	0x30,0x31,0x32,0x33	Table for conversion
	DATABYTE	0x34,0x35,0x36,0x37	to ASCII code.
	DATABYTE	0x38,0x39,0x41,0x42	
	DATABYTE	0x43,0x44,0x45,0x46	

Figure 3.12 A RISC-style program for Example 3.4.

Example 3.5 Problem: Consider the task described in Example 3.1. Assume that the timer circuit includes a 32-bit up/down counter driven by a 100-MHz clock. The counter can be set to count from a specified initial count value. The timer I/O interface is shown in Figure 3.14. It contains four registers.

- TIM_STATUS indicates the current status of the timer where:
 - The TON bit is set to 1 when the counter is running.
 - The ZERO bit is set to 1 when the counter reaches the count of zero.
 - The TIRQ bit is set to 1 when the timer raises an interrupt request, which happens when the counter contents reach zero and the timer interrupts are enabled.

	Move	R2, BINARY	Load the binary number.
	Move	R3, #8	R3 is a digit counter that is set to 8.
	Move	R4, #HEX	R4 points to the hex digits.
LOOP:	RotateL	R2, #4	Rotate the high-order digit into low-order position.
	Move	R5, R2	
	And	R5, #0xF	Extract next digit.
	MoveByte	(R4)+, TABLE(R5)	Get ASCII code for the digit and store it in HEX number location.
	Subtract	R3, #1	Decrement the digit counter.
	Branch>0	LOOP	Loop back if not the last digit.
DISPLAY:	Move	R3, #8	
	Move	R4, #HEX	
DLOOP:	TestBit	DISP_STATUS, #2	Wait for display to become ready.
	Branch=0	DLOOP	
	MoveByte	DISP_DATA, (R4)+	Send next character to display.
	Subtract	R3, #1	Decrement the counter.
	Branch>0	DLOOP	Loop until all characters displayed.
	next instruction		
HEX:	ORIGIN	1000	
	RESERVE	8	Space for ASCII-encoded digits.
TABLE:	DATABYTE	0x30,0x31,0x32,0x33	Table for conversion to ASCII code.
	DATABYTE	0x34,0x35,0x36,0x37	
	DATABYTE	0x38,0x39,0x41,0x42	
	DATABYTE	0x43,0x44,0x45,0x46	

Figure 3.13 A CISC-style program for Example 3.4.

The action of reading the status register automatically clears the ZERO and TIRQ bits to 0.

- TIM_CONT controls the mode of operation, where:
 - The UP bit is set to 1 to cause the counter to count by incrementing its contents; when this bit is cleared to zero, the counter contents are decremented.
 - The FREE bit is set to 1 to cause a continuously running mode, where the counter is automatically reloaded with the initial count value whenever the actual count reaches zero.
 - The RUN bit is set to 1 to cause the counter to count; it is cleared to 0 to stop the counter.
 - The TIE bit is set to 1 to enable timer interrupts.
- TIM_INIT holds the initial count value.
- TIM_COUNT holds the current count value.

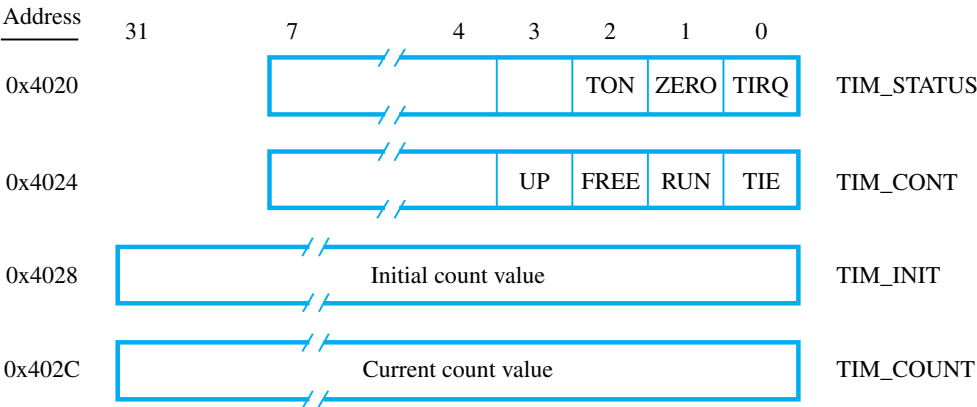


Figure 3.14 Registers in the timer interface.

Write a program to implement the desired task. Use the processor control registers depicted in Figure 3.7.

Solution: To obtain an interrupt request every ten seconds, it is necessary to count 10^9 clock cycles. This can be accomplished by writing this value into the TIM_INIT register, and then making the counter decrement its count and raise an interrupt when the count reaches zero. The value 10^9 can be represented by the hexadecimal number 3B9ACA00. To achieve the desired operation the FREE, RUN, and TIE bits must be set to 1, while the UP bit must be equal to 0.

Using the scheme outlined in Figure 3.9, we can implement the required task using a RISC-style program shown in Figure 3.15. Note that the initial count, which is a 32-bit immediate value, is loaded into R2 using the approach explained in Section 2.9.

Figure 3.16 gives a CISC-style program that uses the scheme outlined in Figure 3.11. In this case, the 32-bit immediate operand can be specified in a single instruction.

Example 3.6 Problem: A commonly used output device in digital systems is a seven-segment display, depicted in Figure 3.17. The device consists of seven independent segments which can be illuminated by applying electrical signals to them. Assume that each segment is illuminated when a logic value 1 is applied to it. The figure shows the bit patterns needed to display numbers 0 to 9.

Write a program that displays the number represented by an ASCII-encoded character stored in memory location DIGIT at address 0x800. Assume that the display has an I/O interface consisting of an eight-bit data register, SEVEN, where the segments *a* to *g* are connected to bits SEVEN₆₋₀. Let the bit SEVEN₇ be equal to 0. Also, assume that the address of register SEVEN is 0x4030. If the ASCII code in location DIGIT represents a

Interrupt handler

ILOC:	Subtract	SP, SP, #8	Save registers.
	Store	LINK_reg, 4(SP)	
	Store	R2, (SP)	
	MoveControl	R2, IPENDING	Check contents of IPENDING.
	And	R2, R2, #8	Check if request from timer.
	Branch_if_[R2]=0	NEXT	
	LoadByte	R2, TIM_STATUS	Clear TIRQ and ZERO bits.
NEXT:	Call	DISPLAY	Call the DISPLAY routine.
	...		Check for other interrupts.
	Load	R2, (SP)	Restore registers.
	Load	LINK_reg, 4(SP)	
	Add	SP, SP, #8	
	Return-from-interrupt		

Main program

START:	...		Set up parameters for ISRs.
	OrHigh	R2, R0, #0x3B9A	Prepare the initial
	Or	R2, R2, #0xCA00	count value.
	Store	R2, TIM_INIT	Set the initial count value.
	Move	R2, #7	Set the timer to free run
	StoreByte	R2, TIM_CONT	and enable interrupts.
	MoveControl	R2, IENABLE	
	Or	R2, R2, #8	Enable timer interrupts in
	MoveControl	IENABLE, R2	the processor control register.
	MoveControl	R2, PS	
COMPUTE:	Or	R2, R2, #1	
	MoveControl	PS, R2	Set interrupt-enable bit in PS.
	next instruction		

Figure 3.15 A RISC-style program for Example 3.5.

character that is not a number in the range 0 to 9, then the display should be blank, where all segments are turned off.

Solution: A look-up table can be used to hold the seven-segment bit patterns that correspond to the numbers 0 to 9. The ASCII-encoded digit is converted into a four-bit number that is used as an index into the table, by using the AND operation. Also, it is necessary to check that the high-order four bits of ASCII code are 0011. Note that all three addresses DIGIT, SEVEN, and TABLE can be represented in 16 bits.

Figures 3.18 and 3.19 give possible RISC- and CISC-style programs, respectively.

Interrupt handler

ILOC:	Move	– (SP), R2	Save registers.
	Move	– (SP), LINK_reg	
	MoveControl	R2, IPENDING	Check contents of IPENDING.
	TestBit	R2, #3	Check if request from timer.
	Branch=0	NEXT	
	MoveByte	R2, TIM_STATUS	Clear TIRQ and ZERO bits.
NEXT:	Call	DISPLAY	Call the DISPLAY routine.
	...		Check for other interrupts.
	Move	LINK_reg, (SP)+	Restore registers.
	Move	R2, (SP)+	
	Return-from-interrupt		

Main program

START:	...		Set up parameters for ISRs.
	Move	TIM_INIT, #0x3B9ACA00	Set the initial count value.
	MoveByte	TIM_CON, #7	Set the timer to free run and enable interrupts.
	MoveControl	R2, IENABLE	
	Or	R2, #8	Enable timer interrupts in the processor control register.
	MoveControl	IENABLE, R2	
	MoveControl	R2, PS	
	Or	R2, #1	
COMPUTE:	MoveControl	PS, R2	Set interrupt-enable bit in PS.
	next instruction		

Figure 3.16 A CISC-style program for Example 3.5.

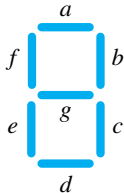
	Number	a	b	c	d	e	f	g
	0	1	1	1	1	1	1	0
	1	0	1	1	0	0	0	0
	2	1	1	0	1	1	0	1
	3	1	1	1	1	0	0	1
	4	0	1	1	0	0	1	1
	5	1	0	1	1	0	1	1
	6	1	0	1	1	1	1	1
	7	1	1	1	0	0	0	0
	8	1	1	1	1	1	1	1
	9	1	1	1	1	0	1	1

Figure 3.17 Seven-segment display.

DIGIT	EQU	0x800	Location of ASCII-encoded digit.
SEVEN	EQU	0x4030	Address of 7-segment display.
	LoadByte	R2, DIGIT	Load the ASCII-encoded digit.
	And	R3, R2, #0xF0	Extract high-order bits of ASCII.
	And	R2, R2, #0x0F	Extract the decimal number.
	Move	R4, #0x30	Check if high-order bits of
	Branch_if_[R3]=[R4]	HIGH3	ASCII code are 0011.
	Move	R2, #0x0F	Not a digit, display a blank.
HIGH3:	LoadByte	R5, TABLE(R2)	Get the 7-segment pattern.
	StoreByte	R5, SEVEN	Display the digit.
	ORIGIN	0x1000	
TABLE:	DATABYTE	0x7E,0x30,0x6D,0x79	Table that contains
	DATABYTE	0x33,0x5B,0x5F,0x70	the necessary
	DATABYTE	0x7F,0x7B,0x00,0x00	7-segment patterns.
	DATABYTE	0x00,0x00,0x00,0x00	

Figure 3.18 A RISC-style program for Example 3.6.

DIGIT	EQU	0x800	Location of ASCII-encoded digit.
SEVEN	EQU	0x4030	Address of 7-segment display.
	Move	R2, DIGIT	Load the ASCII-encoded digit.
	Move	R3, R2	
	And	R3, #0xF0	Extract high-order bits of ASCII.
	And	R2, #0x0F	Extract the decimal number.
	CompareByte	R3, #0x30	Check if high-order bits of
	Branch=0	HIGH3	ASCII code are 0011.
	Move	R2, #0x0F	Not a digit, display a blank.
HIGH3:	MoveByte	SEVEN, TABLE(R2)	Display the digit.
	ORIGIN	0x1000	
TABLE:	DATABYTE	0x7E,0x30,0x6D,0x79	Table that contains
	DATABYTE	0x33,0x5B,0x5F,0x70	the necessary
	DATABYTE	0x7F,0x7B,0x00,0x00	7-segment patterns.
	DATABYTE	0x00,0x00,0x00,0x00	

Figure 3.19 A CISC-style program for Example 3.6.

PROBLEMS

- 3.1** [E] The input status bit in an interface circuit is cleared as soon as the input data register is read. Why is this important?
- 3.2** [E] Write a program that displays the contents of ten bytes of the main memory in hexadecimal format on a line of a display device. The ten bytes start at location LOC in the memory, and there are two hex characters per byte. The contents of successive bytes should be separated by a space when displayed.
- 3.3** [E] What is the difference between a subroutine and an interrupt-service routine?
- 3.4** [E] In the first And instruction in Figure 3.4 the immediate value 2 is used when checking the KIN flag, but in Figure 3.5 the immediate value 1 is used in the first TestBit instruction when checking the same flag. Explain the difference.
- 3.5** [D] A computer is required to accept characters from the keyboard input of 20 terminals. The main memory area to be used for storing data for each terminal is pointed to by a pointer $PNTR_n$, where $n = 1$ through 20. Input data must be collected from the terminals while another program PROG is being executed. This may be accomplished in one of two ways:
- (a) Every T seconds, program PROG calls a polling subroutine POLL. This subroutine checks the status of each of the 20 terminals in sequence and transfers any input characters to the memory. Then it returns to PROG.
 - (b) Whenever a character is ready in any of the interface buffers of the terminals, an interrupt request is generated. This causes the interrupt routine INTERRUPT to be executed. INTERRUPT polls the status registers to find the first ready character, transfers it, and then returns to PROG.
- Write the routines POLL and INTERRUPT. Let the maximum character rate for any terminal be c characters per second, with an average rate equal to rc , where $r \leq 1$. In method (a), what is the maximum value of T for which it is still possible to guarantee that no input characters will be lost? What is the equivalent value for method (b)? Estimate, on the average, the percentage of time spent in servicing the terminals for methods (a) and (b), for $c = 100$ characters per second and $r = 0.01, 0.1, 0.5$, and 1. Assume that POLL takes 800 ns to poll all 20 devices and that an interrupt from a device requires 200 ns to process.
- 3.6** [E] In Figure 3.9, the interrupt-enable bit in the PS is set last in the START section of the Main program. Why? Does the order matter for earlier operations in START? Why or why not?
- 3.7** [E] Even if multiple interrupt requests are pending, only one request will be handled for each entry into ILOC in Figure 3.9. True or false? Explain.
- 3.8** [E] A user program could check for a zero divisor immediately preceding each division operation, and then take appropriate action without invoking the OS. Give reasons why this may or may not be preferable to allowing an exception interrupt to occur on an actual divide by zero situation in a user program.

- 3.9** [M] Assume that a memory location BINARY contains a 16-bit pattern. It is desired to display these bits as a string of 0s and 1s on a display device that has the interface depicted in Figure 3.3. Write a RISC-style program that accomplishes this task.
- 3.10** [M] Write a CISC-style program for the task in Problem 3.9.
- 3.11** [E] Modify the program in Figure 3.18 if the address of TABLE is 0x10100.
- 3.12** [E] Modify the program in Figure 3.19 if the address of TABLE is 0x10100.
- 3.13** [M] Using the seven-segment display in Figure 3.17 and the timer interface registers in Figure 3.14, write a RISC-style program that flashes decimal digits in the repeating sequence 0, 1, 2, . . . , 9, 0, Each digit is to be displayed for one second. Assume that the counter in the timer circuit is driven by a 100-MHz clock.
- 3.14** [M] Write a CISC-style program for the task in Problem 3.13.
- 3.15** [D] Using two 7-segment displays of the type shown in Figure 3.17, and the timer interface registers in Figure 3.14, write a RISC-style program that flashes the repeating sequence of numbers 0, 1, 2, . . . , 98, 99, 0, Each number is to be displayed for one second. Assume that the counter in the timer circuit is driven by a 100-MHz clock.
- 3.16** [D] Write a CISC-style program for the task in Problem 3.15.
- 3.17** [D] Write a RISC-style program that computes wall clock time and displays the time in hours (0 to 23) and minutes (0 to 59). The display consists of four 7-segment display devices of the type shown in Figure 3.17. A timer circuit that has the interface registers given in Figure 3.14 is available. Its counter is driven by a 100-MHz clock.
- 3.18** [D] Write a CISC-style program for the task in Problem 3.17.
- 3.19** [M] Write a RISC-style program that displays the name of the user backwards. The program should display a prompt requesting that the characters in the user's name be entered on the keyboard, followed by the carriage return (CR). The program should accept a sequence of characters and store them in the main memory. It should then display a message to indicate that the user's name will be displayed backwards, followed by the display of the characters from the user's name in reverse order.
- 3.20** [M] Write a CISC-style program for the task in Problem 3.19.
- 3.21** [M] Write a RISC-style program that determines whether a word entered by a user on the keyboard is a palindrome, i.e., a word that is same when its characters are written in normal and reverse order. The program should display a prompt requesting that the user enter the characters of an arbitrary word on the keyboard, followed by the carriage return (CR). The program should read the characters and store them in the main memory. It should then analyze the word to determine whether it is a palindrome. Finally, the program should display a message to indicate the result of the analysis.
- 3.22** [M] Write a CISC-style program for the task in Problem 3.21.

- 3.23** [D] Write a RISC-style program that displays a string of characters centered horizontally on a standard 80-character line and enclosed in a box, as shown below:

```
+-----+  
|sample text|  
+-----+
```

The string of characters is located in the main memory beginning at address `STRING`. There is a NUL control character (value 0) at the end of the string of characters. If the string has more than 78 characters (including spaces), the program should truncate the displayed string to 78 characters. The program for determining the length of a character string in Example 2.1 can be adapted as a subroutine for use by the program in this problem. Assume that the display device has the interface depicted in Figure 3.3.

- 3.24** [D] Write a CISC-style program for the task in Problem 3.23.

- 3.25** [D] Write a RISC-style program that displays a long sequence of text encoded in ASCII characters with automatic wraparound to fit within 80-character lines. Before displaying the next word, the program must determine whether there is sufficient space remaining on the line. If not, the word should appear at the beginning of the next line. The display process must continue until the NUL control character (value 0) is reached at the end of the sequence of characters to be displayed. Assume that the sequence of characters uses no control characters other than the NUL character at the end, hence words are separated only by a space character. Assume that the display device has the interface depicted in Figure 3.3.

- 3.26** [D] Write a CISC-style program for the task in Problem 3.25.